Pulse View – Real-Time Hospital Report

This project is a comprehensive Hospital Database Management System designed to digitally manage all critical aspects of a hospital's operations. The goal is to eliminate the inefficiencies of manual recordkeeping by using a structured relational database to handle patients, staff, rooms, appointments, treatments, and more.

Objectives:

- Efficiently store, retrieve, and manage hospital data
- Support complex queries related to patient care, staff assignments, room availability, and medical procedures
- Ensure data integrity and normalization across interconnected entities

---

Key Features & Components:

- Relational Schema: A normalized database structure with 15+ interrelated tables including:
  - Physician, Nurse, Patient, Appointment, Medication, Room, Procedure, On_Call, and more.
- Data Integrity & Relationships:
  - Implemented primary and foreign key constraints to enforce data relationships
  - Logical constraints (e.g., registered nurses, certified physicians, room availability)
- Advanced SQL Querying:
  - A complete set of 39 SQL queries designed to solve real-life hospital use cases, such as:
    - Listing available and unavailable rooms by block/floor
    - Fetching nurse and physician schedules
    - Validating certification for medical procedures
    - Identifying unregistered nurses or unaffiliated physicians
    - Detecting inconsistencies in treatment assignments

Table Creations

In total 15 tables were created that define all of the entities and their relationship in the Hospital system as specified in the ERD then filled with dummy data for the sake of testing and visualization of output.

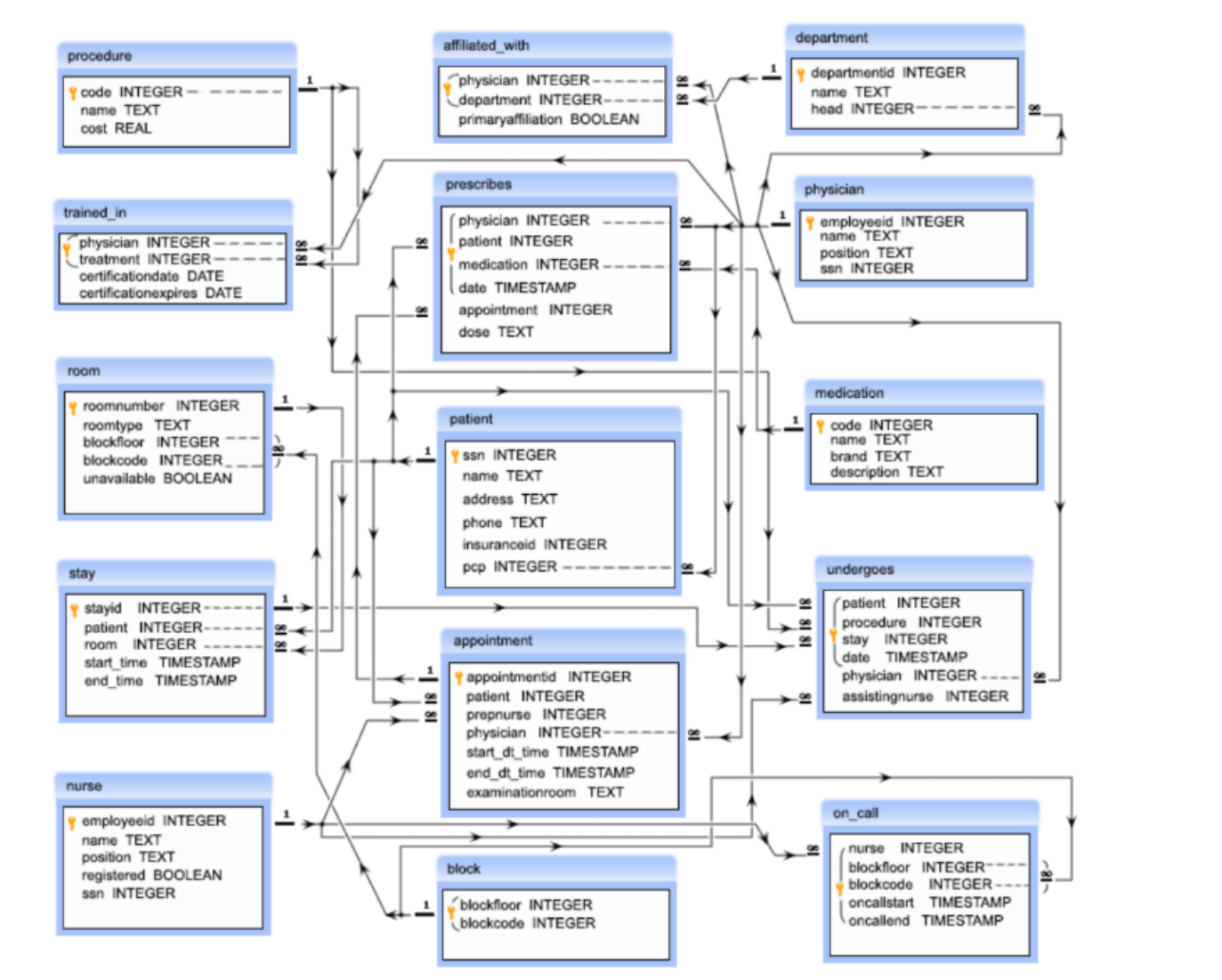


Table Creations

Department Table

```
CREATE TABLE Department (
    department_id INTEGER PRIMARY KEY,
    dep_name TEXT NOT NULL,
    head INTEGER NOT NULL,
    FOREIGN KEY (head) REFERENCES Physician(employee_id)
);
```

The **Department** table defines each department with a unique ID and name, while establishing a foreign key relationship with the **Physician** table to assign department heads.
This ensures data integrity by linking each department to an existing physician. The structure supports efficient queries and maintains consistency across the hospital database.

	department_id	dep_name	head
1	1	General Medicine	4
2	2	Surgery	3
3	3	Psychiatry	2
4	4	Cardiology	1
5	5	Administration	5
6	6	Neurology	12
7	7	Oncology	8
8	8	Immunology	11
9	9	Pediatrics	7
10	10	Orthopedics	14
11	11	Dermatology	17
12	12	Urology	13
13	13	Radiology	10
14	14	Plastic Surgery	16
15	15	Internal Medicine	18
16	16	Pathology	19
17	17	Rehabilitation	9
18	18	Endocrinology	6
19	19	Emergency M...	20
20	20	Family Medicine	15

Table Creations

TABLE Affiliated_wit

```
CREATE TABLE Affiliated_with (
    physician INTEGER NOT NULL,
    department INTEGER NOT NULL,
    primary_affiliation BOOLEAN NOT NULL,
    PRIMARY KEY (physician, department),
    FOREIGN KEY (physician) REFERENCES Physician(employee_id),
    FOREIGN KEY (department) REFERENCES Department(department_id)
);
```

The **Affiliated_with** table manages **physician–department** relationships, using a Boolean flag for primary affiliation and enforcing data integrity through composite and foreign keys.

	physician	department	primary_affiliation
1	1	1	1
2	1	4	0
3	2	1	0
4	2	3	1
5	3	2	1
6	3	6	0
7	4	1	1
8	4	7	0
9	5	5	1
10	6	3	1
11	7	9	1
12	8	2	0
13	8	7	1

Table Creations

TABLE ON_Call

```
CREATE TABLE On_Call (
    nurse INTEGER NOT NULL,
    block_floor INTEGER NOT NULL,
    block_code INTEGER NOT NULL,
    on_call_start TIMESTAMP NOT NULL,
    on_call_end TIMESTAMP NOT NULL,
    PRIMARY KEY (nurse, block_floor, block_code, on_call_start, on_call_end),
    FOREIGN KEY (nurse) REFERENCES Nurse(employee_id),
    FOREIGN KEY (block_floor, block_code) REFERENCES Block(block_floor, block_code)
);
```

	nurse	block_floor	block_code	on_call_start	on_call_end
1	101	1	101	2023-04-25 08:00:00.0000000	2023-04-25 16:00:00.0000000
2	102	1	102	2023-04-25 16:00:00.0000000	2023-04-26 00:00:00.0000000
3	103	1	101	2023-05-04 16:00:00.0000000	2023-05-05 00:00:00.0000000
4	104	2	201	2023-04-26 08:00:00.0000000	2023-04-26 16:00:00.0000000
5	105	2	202	2023-04-26 16:00:00.0000000	2023-04-27 00:00:00.0000000
6	106	2	203	2023-04-27 08:00:00.0000000	2023-04-27 16:00:00.0000000
7	107	3	301	2023-04-27 16:00:00.0000000	2023-04-28 00:00:00.0000000
8	108	3	302	2023-04-28 08:00:00.0000000	2023-04-28 16:00:00.0000000
9	109	3	303	2023-04-28 16:00:00.0000000	2023-04-29 00:00:00.0000000
10	110	4	401	2023-04-29 08:00:00.0000000	2023-04-29 16:00:00.0000000
11	111	4	402	2023-04-29 16:00:00.0000000	2023-04-30 00:00:00.0000000
12	112	4	403	2023-04-30 08:00:00.0000000	2023-04-30 16:00:00.0000000
13	113	4	404	2023-04-30 16:00:00.0000000	2023-05-01 00:00:00.0000000

The **On_Call** table tracks nurse duty shifts by block and floor, using timestamps for start and end times. Composite keys and foreign constraints ensure accurate scheduling and location referencing.

Query 1

Find all information of nurses who are yet to be registered

Finds all nurses whose registered column value is False

SQL code:

```
SELECT *
FROM Nurse
WHERE registered = false;
```

	employee_id [PK] integer	nurse_name text	nurse_position text	registered boolean	ssn integer
1	103	Paul Flowers	Nurse	false	888888888
2	105	Todd Quinlan	Junior Nurse	false	101010101

Query 2

Find names of nurses who are department heads

The department table contains each department with the employee id that corresponds to its head.
This code finds the intersection between them to output the names of the nurses whose ids are in the head column of the department table, but since according to this schema only doctors are allowed to be department heads (as seen in the relationship in the ERD) the output is empty

SQL code:

```
SELECT n.nurse_name, d.dep_name AS department
FROM Nurse n
JOIN Department d ON n.employee_id = d.head;
```

	nurse_name text	department text

Query 3

Find names of physicians who are department heads

Now this code find the physician heads of each department. It works in the same way as the last query except this time it will display output as there are in fact heads of departments who are doctors.

SQL code:

```
SELECT p.physician_name, d.dep_name AS department_name
FROM Physician p
JOIN Department d ON p.employee_id = d.head;
```

	physician_name text	department_name text
1	Percival Cox	General Medicine
2	Christopher Turk	Surgery
3	Elliot Reid	Psychiatry
4	John Dorian	Cardiology
5	Bob Kelso	Administration

Query 4

Count patients who have taken at least one appointment

This code uses the aggregate function Count to count all the patients who have booked at least one appointment .

SQL code:

```
SELECT COUNT(DISTINCT a.patient) AS patients_with_appointments
FROM Appointment a;
```

	patients_with_appointments bigint
1	5

Query 5

Find floor and block for room number 212

This code displays the floor and block numbers for all rooms in table Room whose room number = 12 (condition applied). Of course since room number is a primary key then only one room has the number 212.

SQL code:

```
SELECT r.room_number, r.block_floor AS floor, r.block_code AS block
FROM Room r
WHERE r.room_number = 212;
```

	room_number [PK] integer	floor integer	block integer
1	212	2	202

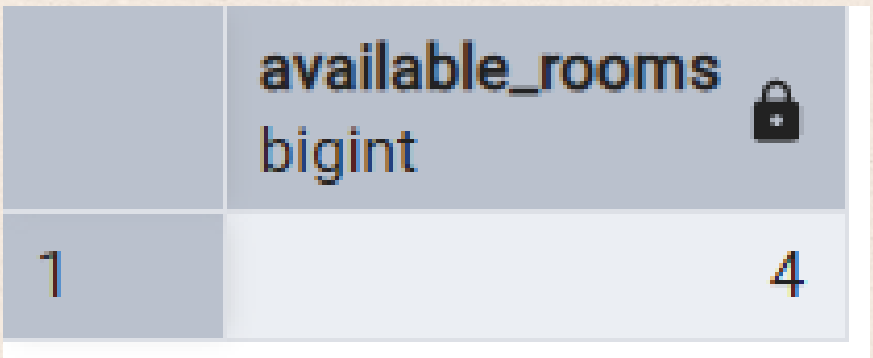
Query 6

Count the number of available rooms

This code uses the aggregate function to see all the rooms where the condition unavailable = false is met and then counts the number of them.

SQL code:

```
SELECT COUNT(*) AS available_rooms
FROM Room
WHERE unavailable = false;
```



	available_rooms bigint
1	4

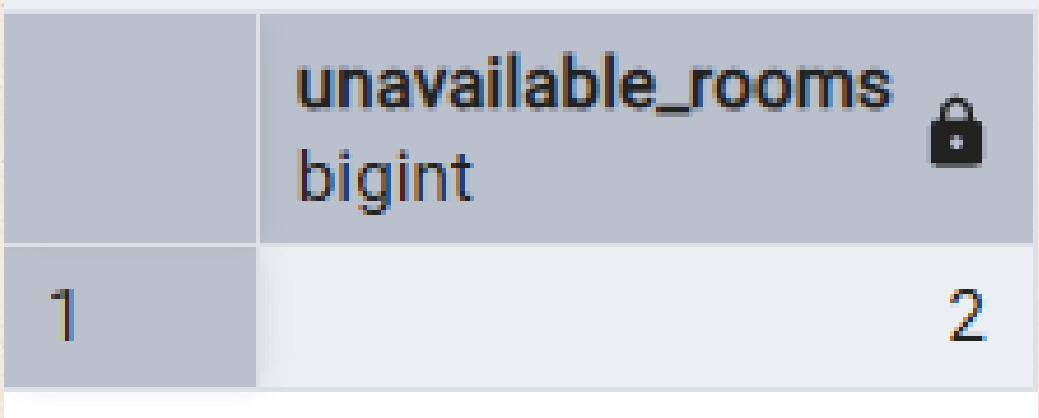
Query 7

Count the number of unavailable rooms

The same as the last Query except this is for the unavailable rooms (so condition will be WHERE unavailable = true)

SQL code:

```
SELECT COUNT(*) AS unavailable_rooms
FROM Room
WHERE unavailable = true;
```



	unavailable_rooms bigint
1	2

Query 8

Get physician names and their affiliated departments

Joins three tables: {hysiscian, Department and the relationship tale Affiliated_with) to find the department to which each physician is affiliated.

SQL code:

```
SELECT p.physician_name, d.dep_name AS department
FROM Physician p
JOIN Affiliated_with a ON p.employee_id = a.physician
JOIN Department d ON a.department = d.department_id
WHERE a.primary_affiliation = true;
```

	physician_name text	department text
1	John Dorian	General Medicine
2	Elliot Reid	General Medicine
3	Christopher Turk	Surgery
4	Percival Cox	General Medicine
5	Bob Kelso	Administration

Query 9

Get physicians trained for special treatments

Finds special treatment each doctor specializes in

SQL code:

```
SELECT DISTINCT p.physician_name, pi.procedure_name AS special_treatment
FROM Physician p
JOIN Trained_in t ON p.employee_id = t.physician
JOIN Procedure_Info pi ON t.treatment = pi.code;
```

	physician_name text	special_treatment text
1	Christopher Turk	Colonoscopy
2	Elliot Reid	Psych Evaluation
3	John Dorian	Angioplasty
4	Percival Cox	MRI Scan
5	Christopher Turk	Appendectomy

Query 10

Get physicians with department who are yet to be affiliated

Finds all physicians who are not affiliated with any departments

SQL code:

```
SELECT p.physician_name
FROM Physician p
LEFT JOIN Affiliated_with a ON p.employee_id = a.physician
WHERE a.physician IS NULL;
```

	physician_name text
1	Molly Clock

Query 11

Get physicians who are not specialized

Uses left join to find any employees who do not have a corresponding value in the specialization column

SQL code:

```
SELECT p.physician_name
FROM Physician p
LEFT JOIN Trained_in t ON p.employee_id = t.physician
WHERE t.physician IS NULL;
```

	physician_name text
1	Bob Kelso
2	Molly Clock

Query 12

Get patients with their primary care physicians

In each patient row there is the id of the doctor who is the primary physician of that patient, this code joins the two tables on the physician"s employee id to find the name of the primary care physician

SQL code:

```
SELECT pt.patient_name AS patient, p.physician_name AS primary_physician
FROM Patient pt
JOIN Physician p ON pt.pcp = p.employee_id;
```

	patient text	primary_physician text
1	John Smith	John Dorian
2	Jane Doe	Elliot Reid
3	Robert Johns...	Christopher Turk
4	Sarah Williams	Percival Cox
5	Michael Brown	John Dorian
6	Lisa Ray	Elliot Reid

13)Write a query in SQL to find the name of the patients and the number of physicians they have taken appointment.

```sql
SELECT
    p.patient_name,
    COUNT(DISTINCT a.physician) AS number_of_physicians
FROM
    Patient p
JOIN
    Appointment a ON p.ssn = a.patient
GROUP BY
    p.ssn, p.patient_name
ORDER BY
    number_of_physicians DESC;
```

Output

patient_name	number_of_physicians
John Smith	1
Jane Doe	1
Robert Johnson	1
Sarah Williams	1
Michael Brown	1

This query identifies patients and counts the distinct physicians they have consulted through appointments. It joins the Patient and Appointment tables using the patient's SSN, groups results by patient, and orders them by the number of physicians in descending order. This is useful for a report to analyze patient-physician interactions, highlighting patients who consult multiple specialists, which may indicate complex medical needs.

14)Write a query in SQL to count number of unique patients who got an appointment for examination room C.

```sql
SELECT
    COUNT(DISTINCT patient) AS unique_patients_in_room_C
FROM
    Appointment
WHERE
    examination_room LIKE 'C%';
```

Output

unique_patients_in_room_C
1

This query counts the number of unique patients who have appointments in examination rooms starting with 'C'. It uses the Appointment table and filters by room prefix. This metric is valuable for assessing room utilization in a specific department or wing, aiding facility management in resource allocation.

15)Write a query in SQL to find the name of the patients and the number of the room where they have to go for their treatment.

```sql
SELECT
    p.patient_name,
    a.examination_room AS treatment_room,
    a.start_dt_time AS appointment_time
FROM
    Patient p
JOIN
    Appointment a ON p.ssn = a.patient
ORDER BY
    a.start_dt_time;
```

Output

patient_name	treatment_room	appointment_time
John Smith	A101	2023-04-25 10:00:00
Jane Doe	B202	2023-04-25 11:00:00
Robert Johnson	C303	2023-04-26 09:00:00
Sarah Williams	A101	2023-04-27 14:00:00
Michael Brown	B202	2023-04-28 15:00:00

This query lists patients, their assigned examination rooms, and appointment times for treatments. It joins Patient and Appointment tables, ordered by appointment time. This is essential for scheduling reports, ensuring patients and staff are aware of treatment locations and times.

16)Write a query in SQL to find the name of the nurses and the room scheduled, where they will assist the physicians.

```sql
SELECT
    n.nurse_name,
    a.examination_room AS scheduled_room,
    a.start_dt_time AS appointment_time,
    ph.physician_name AS assisting_physician
FROM
    Nurse n
JOIN
    Appointment a ON n.employee_id = a.prep_nurse
JOIN
    Physician ph ON a.physician = ph.employee_id
WHERE
    a.prep_nurse IS NOT NULL
ORDER BY
    a.start_dt_time;
```

Output

nurse_name	scheduled_room	appointment_time	assisting_physician
Carla Espinosa	A101	2023-04-25 10:00:00	John Dorian
Laverne Roberts	B202	2023-04-25 11:00:00	Elliot Reid
Shirley	A101	2023-04-27 14:00:00	Percival Cox

This query details nurses, their assigned examination rooms, appointment times, and the physicians they assist. It joins Nurse, Appointment, and Physician tables, filtering for non-null nurse assignments. This helps in staff coordination and workload distribution reports.

17)Write a query in SQL to find the name of the patients who taken the appointment on the 25th of April at 10 am, and also display their physician, assisting nurses and room no.

```
SELECT p.patient_name, ph.physician_name, n.nurse_name, a.examination_room, a.start_dt_tim
FROM Appointment a
JOIN Patient p ON a.patient = p.ssn
JOIN Physician ph ON a.physician = ph.employee_id
LEFT JOIN Nurse n ON a.prep_nurse = n.employee_id
WHERE a.start_dt_time = '2023-04-25 10:00:00';
```

Output

patient_name	physician_name	nurse_name	examination_room	start_dt_time
John Smith	John Dorian	Carla Espinosa	A101	2023-04-25 10:00:00

This query retrieves details of appointments on April 25, 2023, at 10 AM, including patient names, physicians, assisting nurses, and room numbers. It uses precise date and time filtering. This is critical for auditing specific time slots or investigating scheduling conflicts.

18)Write a query in SQL to find the name of patients and their physicians who does not require any assistance of a nurse.

```sql
SELECT
    p.patient_name,
    ph.physician_name,
    a.examination_room,
    a.start_dt_time AS appointment_time
FROM
    Appointment a
JOIN
    Patient p ON a.patient = p.ssn
JOIN
    Physician ph ON a.physician = ph.employee_id
WHERE
    a.prep_nurse IS NULL;
```

Output

patient_name	physician_name	examination_room	appointment_time
Robert Johnson	Christopher Turk	C303	2023-04-26 09:00:00
Michael Brown	John Dorian	B202	2023-04-28 15:00:00

This query identifies appointments where no nurse assistance is required, listing patients, physicians, rooms, and times. It filters for null prep_nurse values. This is useful for understanding physician-led procedures and optimizing nurse scheduling.

19)Write a query in SQL to find the name of the patients, their treating physicians and medication

```sql
SELECT
    p.patient_name,
    ph.physician_name AS treating_physician,
    m.med_name AS medication,
    m.brand,
    pr.date AS prescribed_date,
    pr.dose
FROM
    Patient p
JOIN
    Prescribes pr ON p.ssn = pr.patient
JOIN
    Physician ph ON pr.physician = ph.employee_id
JOIN
    Medication m ON pr.medication = m.code
ORDER BY
    p.patient_name, pr.date DESC;
```

Output

patient_name	treating_physician	medication	brand	prescribed_date	dose
Jane Doe	Elliot Reid	Ibuprofen	Advil	2023-04-25 11:20:00	200mg every 8 hours
John Smith	John Dorian	Paracetamol	Tylenol	2023-04-25 10:15:00	500mg every 6 hours
Lisa Ray	Elliot Reid	Sertraline	Zoloft	2023-04-29 09:00:00	50mg daily
Robert Johnson	Christopher Turk	Lisinopril	Zestril	2023-04-26 09:30:00	10mg daily
Sarah Williams	Percival Cox	Metformin	Glucophage	2023-04-27 14:10:00	500mg twice daily

This query lists patients, their treating physicians, and prescribed medications, including brand, dose, and prescription date. It joins multiple tables and orders by patient and recent prescriptions. This is vital for tracking medication histories and physician prescribing patterns.

20)Write a query in SQL to find the name of the patients who taken an advanced appointment, and also display their physicians and medication.

```
    a.start_dt_time AS appointment_time,
    a.examination_room,
    DATE(a.start_dt_time) - CURRENT_DATE AS days_in_advance
FROM
    Appointment a
JOIN
    Patient p ON a.patient = p.ssn
JOIN
    Physician ph ON a.physician = ph.employee_id
LEFT JOIN
    Prescribes pr ON a.patient = pr.patient
                  AND a.physician = pr.physician
                  AND pr.date <= a.start_dt_time
LEFT JOIN
    Medication m ON pr.medication = m.code
WHERE
    a.start_dt_time > CURRENT_DATE
    AND DATE(a.start_dt_time) - CURRENT_DATE > 7
ORDER BY
    days_in_advance DESC;
```

Output

SQL query successfully executed. However, the result set is empty.

This query identifies future appointments booked at least 7 days in advance, including patient, physician, medication details, and days in advance. It filters for future dates and calculates booking lead time. This helps analyze proactive patient care planning.

21)Write a query in SQL to find the name and medication for those patients who did not take any appointment.

```sql
SELECT
    p.patient_name,
    m.med_name AS medication,
    m.brand,
    pr.date AS prescription_date,
    ph.physician_name AS prescribing_physician
FROM
    Patient p
LEFT JOIN
    Appointment a ON p.ssn = a.patient
LEFT JOIN
    Prescribes pr ON p.ssn = pr.patient
LEFT JOIN
    Medication m ON pr.medication = m.code
LEFT JOIN
    Physician ph ON pr.physician = ph.employee_id
WHERE
    a.appointment_id IS NULL  -- Patients with no appointments
    AND pr.medication IS NOT NULL  -- But have medications prescribed
ORDER BY
    p.patient_name;
```

Output

patient_name	medication	brand	prescription_date	prescribing_physician
Lisa Ray	Sertraline	Zoloft	2023-04-29 09:00:00	Elliot Reid

This query lists patients who have no appointments but have prescribed medications, including medication details and prescribing physicians. It uses left joins to identify non-appointed patients with prescriptions. This is key for identifying patients needing follow-up appointments.

22)Write a query in SQL to count the number of available rooms in each block.

```sql
SELECT
    b.block_floor,
    b.block_code,
    COUNT(r.room_number) AS available_rooms_count
FROM
    Block b
LEFT JOIN
    Room r ON b.block_floor = r.block_floor
           AND b.block_code = r.block_code
           AND r.unavailable = false  -- Only available rooms
GROUP BY
    b.block_floor, b.block_code
ORDER BY
    b.block_floor, b.block_code;
```

Output

block_floor	block_code	available_rooms_count
1	101	1
1	102	1
2	201	1
2	202	1

This query counts available rooms in each block, grouped by floor and block code. It joins Block and Room tables, filtering for available rooms. This is essential for facility management reports to optimize room allocation.

23)Write a query in SQL to count the number of available rooms in each floor.

```sql
SELECT
    block_floor AS floor,
    COUNT(room_number) AS available_rooms_count
FROM
    Room
WHERE
    unavailable = false
GROUP BY
    block_floor
ORDER BY
    block_floor;
```

Output

floor	available_rooms_count
1	2
2	2

This query counts available rooms per floor, using the Room table and filtering for availability. It provides a high-level view of room availability, useful for capacity planning in the report.

24)Write a query in SQL to count the number of available rooms for each block in each floor.

```sql
SELECT
    r.block_floor AS floor,
    r.block_code AS block,
    COUNT(r.room_number) AS available_rooms_count,
    GROUP_CONCAT(r.room_number, ', ') AS available_room_numbers,
    GROUP_CONCAT(r.room_type, ', ') AS room_types
FROM
    Room r
WHERE
    r.unavailable = 0
GROUP BY
    r.block_floor, r.block_code
ORDER BY
    r.block_floor, r.block_code;
```

Output

floor	block	available_rooms_count	available_room_numbers	room_types
1	101	1	301	ICU
1	102	1	303	Surgery
2	201	1	304	Psychiatry
2	202	1	305	Cardiology

This query provides a detailed breakdown of available rooms by floor and block, including room numbers and types. It uses GROUP_CONCAT for aggregated lists. This is critical for detailed facility management and scheduling reports.

25)Write a query in SQL to count the number of unavailable rooms for each block in each floor.

```sql
SELECT
    block_floor AS floor,
    block_code AS block,
    COUNT(room_number) AS unavailable_rooms_count,
    GROUP_CONCAT(room_number, ', ') AS unavailable_room_numbers
FROM
    Room
WHERE
    unavailable = 1
GROUP BY
    block_floor, block_code
ORDER BY
    block_floor, block_code;
```

Output

floor	block	unavailable_rooms_count	unavailable_room_numbers
1	101	1	302
2	202	1	212

This query counts unavailable rooms by floor and block, listing room numbers. It helps identify maintenance or occupancy issues, crucial for facility status reports.

26)Write a query in SQL to find out the floor where the maximum no of rooms are available.

```sql
WITH AvailableRoomsByFloor AS (
    SELECT
        block_floor AS floor,
        COUNT(room_number) AS available_rooms_count
    FROM
        Room
    WHERE
        unavailable = false
    GROUP BY
        block_floor
)
SELECT
    floor,
    available_rooms_count
FROM
    AvailableRoomsByFloor
WHERE
    available_rooms_count = (SELECT MAX(available_rooms_count) FROM AvailableRoomsByFloor)
ORDER BY
    floor;
```

Output

floor	available_rooms_count
1	2
2	2

This query identifies the floor with the most available rooms using a CTE and a subquery to find the maximum count. It's useful for pinpointing high-capacity areas for resource allocation in the report.

27) Write a query in SQL to find out the floor where the minimum no of rooms are availableCreations

---

```sql
SELECT TOP 1 block_floor, COUNT(*) AS available_rooms
FROM Room
WHERE unavailable = 0
GROUP BY block_floor
ORDER BY available_rooms ASC;
```

	block_floor	available_rooms
1	2	2

This SQL query is designed to determine which floor has the lowest number of available rooms in the hospital. It filters the data to include only rooms marked as available **(unavailable = 0),** then uses the **GROUP BY** clause to aggregate the count of available rooms per floor. By ordering the results in ascending order using O**RDER BY COUNT(*) ASC**, and applying **TOP 1**, the query efficiently returns only the floor with the minimum availability, helping hospital administration identify underutilized areas or possible space shortages. This insight is crucial for resource allocation, planning patient accommodations, or expanding room availability where needed.

28) obtain the name of the patients, their block, floor, and room number where they are admitted.

```sql
SELECT p.patient_name, r.block_floor, r.block_code, r.room_number
FROM Stay s
JOIN Patient p ON s.patient = p.ssn
JOIN Room r ON s.room = r.room_number;
```

	patient_name	block_floor	block_code	room_number
1	John Smith	1	101	301
2	Jane Doe	1	102	303
3	Robert Johnson	2	201	304
4	Sarah Williams	2	202	305
5	Michael Brown	2	203	306
6	Lisa Ray	3	301	401
7	David Miller	3	302	402
8	Emily Davis	3	303	403
9	James Wilson	4	401	404
10	Mary Taylor	4	402	405
11	William Moore	4	403	406
12	Patricia Anderson	4	404	407
13	Charles Thomas	5	501	408
14	Linda Jackson	5	502	409
15	Christopher White	5	503	410
16	Barbara Harris	5	504	411
17	Daniel Martin	5	505	412
18	Elizabeth Thomp...	3	304	413
19	Matthew Garcia	2	202	212
20	Susan Martinez	1	101	302

This query retrieves the full spatial admission details of each patient, including their name, the floor and block of the hospital, and the exact room number in which they are currently admitted. It joins the Stay, Patient, and Room tables using the patient's SSN and room number as **foreign keys.** This information is crucial for hospital logistics, enabling better room tracking, navigation, visitor guidance, and real-time updates on patient distribution across blocks and floors.

29) obtain the nurses and the block where they are booked for attending the patients on call.

```
SELECT n.nurse_name, oc.block_floor, oc.block_code
FROM On_Call oc
JOIN Nurse n ON oc.nurse = n.employee_id;
```

	nurse_name	block_floor	block_code
1	Carla Espinosa	1	101
2	Laverne Roberts	1	102
3	Paul Flowers	1	101
4	Shirley	2	201
5	Todd Quinlan	2	202
6	The Todd	2	203
7	Nancy Hall	3	301
8	Tim Blake	3	302
9	Angela Reed	3	303
10	Victor Scott	4	401
11	Rachel Green	4	402
12	Monica Geller	4	403
13	Phoebe Buffay	4	404
14	Ross Geller	5	501
15	Joey Tribbiani	5	502
16	Chandler Bing	5	503

This query provides an overview of the on-call nurse assignments across the hospital's blocks and floors. By joining the On_Call and Nurse tables on the nurse's employee ID, it reveals which nurses are assigned to which blocks for patient care during their on-call shifts. This information is essential for hospital staff scheduling, shift planning, emergency coverage, and ensuring adequate nurse distribution across all units to maintain care quality.

30) Patient name, physician, nurse, treatment, release date, room, floor, block.

This query retrieves a detailed summary of each patient's treatment: including their name, physician, nurse (if any), procedure, discharge date, and room details (number, floor, block). It uses multiple joins and a LEFT JOIN for nurses to ensure complete results even if no nurse assisted. Ideal for discharge reports and care tracking.

```sql
SELECT
    p.patient_name,
    ph.physician_name,
    n.nurse_name,
    pi.procedure_name,
    s.end_time AS release_date,
    r.room_number,
    r.block_floor,
    r.block_code
FROM Undergoes u
JOIN Patient p ON u.patient = p.ssn
JOIN Physician ph ON u.physician = ph.employee_id
LEFT JOIN Nurse n ON u.assisting_nurse = n.employee_id
JOIN Procedure_Info pi ON u.patient_procedure = pi.code
JOIN Stay s ON u.stay = s.stay_id
JOIN Room r ON s.room = r.room_number;
```

	patient_name	physician_name	nurse_name	procedure_name	release_date	room_number	block_floor	block_code
1	John Smith	Elliot Reid	Laverne Roberts	Appendectomy	2023-04-22 15:00:00.0000000	301	1	101
2	John Smith	Christopher Turk	Laverne Roberts	Colonoscopy	2023-04-22 15:00:00.0000000	301	1	101
3	John Smith	John Dorian	Paul Flowers	Angioplasty	2023-04-22 15:00:00.0000000	301	1	101
4	John Smith	Percival Cox	NULL	Psych Evaluation	2023-04-25 18:00:00.0000000	304	2	201
5	Jane Doe	Christopher Turk	Laverne Roberts	Appendectomy	2023-04-23 12:00:00.0000000	303	1	102
6	Jane Doe	Christopher Turk	Paul Flowers	Appendectomy	2023-04-23 12:00:00.0000000	303	1	102
7	Jane Doe	Percival Cox	Shirley	Colonoscopy	2023-04-25 18:00:00.0000000	304	2	201
8	Robert Johnson	Elliot Reid	Laverne Roberts	Appendectomy	2023-04-25 18:00:00.0000000	304	2	201
9	Robert Johnson	Bob Kelso	NULL	Colonoscopy	2023-04-22 15:00:00.0000000	301	1	101
10	Robert Johnson	Elliot Reid	Shirley	Psych Evaluation	2023-04-25 18:00:00.0000000	304	2	201
11	Sarah Williams	John Dorian	Carla Espinosa	Appendectomy	2023-04-22 15:00:00.0000000	301	1	101
12	Sarah Williams	Percival Cox	Shirley	MRI Scan	2023-04-24 19:00:00.0000000	305	2	202
13	Michael Brown	Percival Cox	Shirley	Appendectomy	2023-04-23 12:00:00.0000000	303	1	102
14	Michael Brown	Bob Kelso	NULL	CT Scan	2023-04-26 14:00:00.0000000	306	2	203
15	Lisa Ray	John Dorian	Carla Espinosa	Angioplasty	2023-04-25 18:00:00.0000000	304	2	201
16	Lisa Ray	Molly Clock	Todd Quinlan	Blood Test	2023-04-27 12:00:00.0000000	401	3	301
17	David Miller	Elliot Reid	NULL	Angioplasty	2023-04-23 12:00:00.0000000	303	1	102
18	David Miller	Lisa Cuddy	The Todd	X-Ray	2023-04-26 09:00:00.0000000	402	3	302
19	Emily Davis	Christopher Turk	Paul Flowers	MRI Scan	2023-04-22 15:00:00.0000000	301	1	101
20	Emily Davis	James Wilson	NULL	Ultrasound	2023-04-27 17:00:00.0000000	403	3	303
21	James Wilson	Gregory House	Nancy Hall	Cataract Surgery	2023-04-28 15:00:00.0000000	404	4	401
22	Mary Taylor	Robert Chase	Tim Blake	Gastroscopy	2023-04-29 18:00:00.0000000	405	4	402
23	William Moore	Allison Cameron	NULL	ECG	2023-05-01 10:00:00.0000000	406	4	403
24	Patricia Ander...	Eric Foreman	Angela Reed	Dialysis	2023-05-02 11:00:00.0000000	407	4	404
25	Charles Thomas	Remy Hadley	NULL	Chemotherapy...	2023-05-03 08:00:00.0000000	408	5	501
26	Linda Jackson	Lawrence Kutner	Victor Scott	Vaccination	2023-05-04 14:00:00.0000000	409	5	502
27	Christopher ...	Chi Park	NULL	Skin Biopsy	2023-05-05 16:00:00.0000000	410	5	503
28	Barbara Harris	Chris Taub	Rachel Green	Laparoscopy	2023-05-06 15:00:00.0000000	411	5	504
29	Daniel Martin	Jessica Adams	NULL	Brain MRI	2023-05-07 12:00:00.0000000	412	5	505
30	Elizabeth Tho...	Jeffrey Cole	Monica Geller	Physical Thera...	2023-05-08 18:00:00.0000000	413	3	304
31	Matthew Garcia	Samira Shah	NULL	EEG	2023-05-09 13:00:00.0000000	212	2	202

31) Physicians who performed procedures without being certified.

```sql
-- 31) Physicians who performed procedures without being certified.
SELECT DISTINCT ph.physician_name
FROM Undergoes u
JOIN Physician ph ON u.physician = ph.employee_id
WHERE NOT EXISTS (
    SELECT 1 FROM Trained_in t
    WHERE t.physician = u.physician AND t.treatment = u.patient_procedure
);
```

07 %

Results | Messages

	physician_name
1	Bob Kelso
2	Christopher Turk
3	Elliot Reid
4	John Dorian
5	Noah Daniels
6	Percival Cox

This query is designed to identify a critical compliance issue within the hospital system—physicians who performed medical procedures without being officially certified or trained to do so. By joining the Undergoes and Physician tables and applying a NOT EXISTS subquery that checks for matching entries in the Trained_in table, the query isolates any physician whose name is missing from the relevant training records for the procedure they conducted. This logic ensures that only violations of certification policy are surfaced. Such a report is vital for risk management, regulatory audits, and internal quality control, as it highlights cases that may expose the institution to legal or ethical accountability. Addressing these gaps helps reinforce proper credentialing and ensure patient safety.

32) Name, procedure, date, patient - for uncertified physician procedures

```sql
-- 32) Name, procedure, date, patient - for uncertified physician procedures.
SELECT ph.physician_name, pi.procedure_name, u.date, p.patient_name
FROM Undergoes u
JOIN Physician ph ON u.physician = ph.employee_id
JOIN Procedure_Info pi ON u.patient_procedure = pi.code
JOIN Patient p ON u.patient = p.ssn
WHERE NOT EXISTS (
    SELECT 1 FROM Trained_in t
    WHERE t.physician = u.physician AND t.treatment = u.patient_procedure
);
```

07 %

Results | Messages

	physician_name	procedure_name	date	patient_name
1	Elliot Reid	Appendectomy	2023-05-01 09:00:00.0000000	John Smith
2	Percival Cox	Psych Evaluation	2023-05-12 15:30:00.0000000	John Smith
3	Percival Cox	Colonoscopy	2023-05-07 10:45:00.0000000	Jane Doe
4	Elliot Reid	Appendectomy	2023-05-03 11:00:00.0000000	Robert Johnson
5	Bob Kelso	Colonoscopy	2023-05-08 11:00:00.0000000	Robert Johnson
6	John Dorian	Appendectomy	2023-05-04 12:00:00.0000000	Sarah Williams
7	Percival Cox	Appendectomy	2023-05-05 13:00:00.0000000	Michael Brown
8	Elliot Reid	Angioplasty	2023-05-10 14:00:00.0000000	David Miller
9	Christopher Turk	MRI Scan	2023-05-11 08:30:00.0000000	Emily Davis
10	Noah Daniels	Colonoscopy	2023-05-09 14:10:00.0000000	Susan Martinez

This query provides a detailed audit of uncertified medical procedures, listing each case where a physician performed a procedure without proper certification. It retrieves the physician's name, the procedure name, the date it occurred, and the patient's name. The query uses a combination of JOIN statements across key tables (Undergoes, Physician, Procedure_Info, Patient) and applies a NOT EXISTS filter against the Trained_in table to ensure only non-certified procedures are shown.

This is an essential tool for risk assessment, performance review, and policy enforcement, helping the hospital administration pinpoint specific incidents that may warrant investigation, retraining, or administrative action. It goes a step beyond general summaries by listing each individual occurrence, making it useful for compiling incident-level compliance reports.

33) Physicians who performed procedures after certification expiry.

```sql
-- 33) Physicians who performed procedures after certification expiry.
SELECT DISTINCT ph.physician_name, ph.physician_position
FROM Undergoes u
JOIN Trained_in t ON u.physician = t.physician AND u.patient_procedure = t.treatment
JOIN Physician ph ON u.physician = ph.employee_id
WHERE u.date > t.certificationexpires;
```

107 %

Results | Messages

	physician_name	physician_position
1	Allison Cameron	Immunologist

This query identifies physicians who have performed medical procedures after their official certification had expired. It joins the Undergoes, Trained_in, and Physician tables and filters cases where the date of the procedure (u.date) is later than the certificationexpires field.

By using this query, the hospital can detect critical breaches in certification validity, which could pose legal, ethical, and medical risks. These violations, though possibly unintentional, reflect gaps in credential tracking or renewal processes. The outcome of this query is vital for compliance audits, license management, and ensuring that only properly certified physicians are performing specialized procedures at any given time.

This is an essential tool for risk assessment, performance review, and policy enforcement, helping the hospital administration pinpoint specific incidents that may warrant investigation, retraining, or administrative action. It goes a step beyond general summaries by listing each individual occurrence, making it useful for compiling incident-level compliance reports.

34) Physician, procedure, patient, date when certification expired.

```
SELECT
    ph.physician_name,
    ph.physician_position,
    pi.procedure_name,
    u.date,
    pt.patient_name,
    t.certificationexpires
FROM Undergoes u
JOIN Trained_in t ON u.physician = t.physician AND u.patient_procedure = t.treatment
JOIN Physician ph ON u.physician = ph.employee_id
JOIN Patient pt ON u.patient = pt.ssn
JOIN Procedure_Info pi ON u.patient_procedure = pi.code
WHERE u.date > t.certificationexpires;
```

%

Results | Messages

physician_name	physician_position	procedure_name	date	patient_name	certificationexpires
Allison Cameron	Immunologist	ECG	2023-04-30 09:15:00.0000000	William Moore	2022-07-15

This query delivers a detailed breakdown of each case where a physician performed a procedure after their certification had expired. Unlike the previous summary (Query 33), this query includes not only the physician's name and position, but also the specific procedure, the patient involved, the exact date it occurred, and the expiry date of the certification.

By joining across Undergoes, Trained_in, Physician, Patient, and Procedure_Info, it gives full transparency on certification violations. This level of detail is essential for incident-level compliance review, helping the administration investigate specific breaches and take action where needed—such as initiating retraining, updating certification records, or issuing internal reports. It enhances accountability and contributes directly to risk mitigation and patient safety enforcement.

35) Nurses who have ever been on call in the block of room 122.

```sql
-- 35) Nurses who have ever been on call in the block of room 122.
SELECT DISTINCT n.nurse_name
FROM On_Call oc
JOIN Nurse n ON oc.nurse = n.employee_id
JOIN Room r ON oc.block_floor = r.block_floor AND oc.block_code = r.block_code
WHERE r.room_number = 122;
```

Results | Messages

nurse_name
Tim Blake

This query identifies all nurses who have ever been assigned on-call duty in the same block as room 122. It performs a join between the On_Call, Nurse, and Room tables, using the block's floor and code to match the room's location. By filtering on room_number = 122, the query retrieves only those nurses whose on-call assignments covered the specific block where this room is located.

This insight is useful for tracking staff activity history, especially in the case of patient incidents, audit trails, or when assessing staff familiarity with specific wards or blocks. It supports operational transparency and helps with investigations, performance evaluation, or shift planning.

36) Nurses who have ever been on call in the block of room 122.

```
SELECT p.patient_name, ph.physician_name
FROM Prescribes pr
JOIN Patient p ON pr.patient = p.ssn
JOIN Physician ph ON pr.physician = ph.employee_id
WHERE p.pcp = pr.physician;
```

%

Results | Messages

patient_name	physician_name
John Smith	John Dorian
John Smith	John Dorian
Jane Doe	Elliot Reid
Jane Doe	Elliot Reid
Robert Johnson	Christopher Turk
Robert Johnson	Christopher Turk
Sarah Williams	Percival Cox
Lisa Ray	Elliot Reid

This query identifies patients who were prescribed medication by their own Primary Care Physician (PCP). It joins the Prescribes, Patient, and Physician tables to match each patient with their PCP and filters the results using a WHERE clause that ensures the prescribing doctor is the same as the patient's registered PCP.

This is particularly useful for analyzing the extent of care continuity, where the PCP not only oversees the patient but also takes an active role in prescribing treatments. It supports trust tracking, relationship mapping, and physician-patient engagement metrics within the hospita

37) Patients who underwent procedures > 5000 and their PCP.

```sql
SELECT DISTINCT p.patient_name, ph.physician_name
FROM Undergoes u
JOIN Patient p ON u.patient = p.ssn
JOIN Procedure_Info pi ON u.patient_procedure = pi.code
JOIN Physician ph ON p.pcp = ph.employee_id
WHERE pi.procedure_cost > 5000;
```

Results | Messages

patient_name	physician_name
Barbara Harris	Lawrence Kutner
Charles Thomas	Allison Cameron
David Miller	Bob Kelso
James Wilson	Lisa Cuddy
John Smith	John Dorian
Lisa Ray	Elliot Reid

This query retrieves a list of patients who underwent high-cost medical procedures (above 5000 units of cost), along with the names of their Primary Care Physicians (PCPs). By joining the Undergoes, Patient, Procedure_Info, and Physician tables, it ensures each result reflects a valid procedure linked to the patient's registered doctor.

This query is useful for financial oversight, care cost analysis, and identifying which PCPs are associated with patients requiring major procedures. It can also support resource planning, especially when trying to predict follow-up needs, insurance burdens, or specialized care continuity.

38) Patients with at least two appointments with a registered nurse.

```sql
-- 38) Patients with at least two appointments
SELECT p.patient_name, ph.physician_name
FROM Appointment a
JOIN Nurse n ON a.prep_nurse = n.employee_id
JOIN Patient p ON a.patient = p.ssn
JOIN Physician ph ON p.pcp = ph.employee_id
WHERE n.registered = 1
GROUP BY p.patient_name, ph.physician_name
HAVING COUNT(a.appointment_id) >= 2;
```

Results | Messages

patient_name	physician_name
John Smith	John Dorian

This query identifies patients who had at least two appointments assisted by a registered nurse. By joining the Appointment, Nurse, Patient, and Physician tables, and filtering nurses with registered = 1, it groups the results by patient and their primary care physician (PCP), using HAVING COUNT(...) >= 2 to ensure only repeat cases are returned.

This is especially useful for analyzing patient-nursing interaction, monitoring care consistency, and ensuring that registered professionals are regularly involved in patient appointments–an important indicator of care quality and adherence to staffing regulations.

39) Patients whose PCP is not the head of any department.

```sql
SELECT p.patient_name, ph.physician_name
FROM Patient p
JOIN Physician ph ON p.pcp = ph.employee_id
WHERE ph.employee_id NOT IN (
    SELECT head FROM Department
);
```

7 %

Results | Messages

patient_name	physician_name
Lina Hossam	Nawal Saeed

This query lists patients whose Primary Care Physician (PCP) is not the head of any department. It joins the Patient and Physician tables using the PCP relationship and filters out any physician whose employee_id is listed as a department head in the Department table.

This query is useful for organizational insights, showing which patients are being treated by general or non-leadership medical staff. It can help in balancing workloads, evaluating physician promotion eligibility, or assessing access to department leadership in patient care structures.